Diseño e Implementación de un Compilador E2E para el Lenguaje EasyLang con Flex, Bison y Backend en Ensamblador x86 NASM

Arturo Alejandro Maldonado Orozco*, Juan Luis Cojtin Herrera†, y Sebastián Rodrigo Rodas Pineda‡
Facultad de Ingeniería, Ingeniería en Sistemas, Informática y Ciencias de la Computación
Universidad Mesoamericana
Quetzaltenango, Guatemala
*20arturomaldonado@umes.edu.gt, Carné: 202308110
†juanluis2005@umes.edu.gt, Carné: 202408007
‡srodas477@umes.edu.gt, Carné: 202408060

Resumen—Este artículo describe el diseño, la especificación formal y la implementación de un compilador completo de extremo a extremo (End-to-End) para un lenguaje estructurado e imperativo denominado EasyLang. El sistema abarca el análisis léxico mediante expresiones regulares en Flex, el análisis sintáctico y semántico fundamentado en una gramática libre de contexto procesada con Bison (Yacc), la traducción dirigida por la sintaxis para la generación de Código de Tres Direcciones (TDC/TAC) y un backend dedicado que traduce el código intermedio a ensamblador de 32 bits compatible con el ensamblador libre NASM bajo arquitectura x86/Linux. Se analizan los retos de diseño, tales como la asignación dinámica de temporales, la linearización de estructuras de control anidadas (IF-THEN-ELSE, bucles FOR y DO-WHILE), la declaración compuesta de identificadores en memoria (ALLOCATE) y la administración óptima de registros en el código objeto final. Los resultados demuestran la viabilidad, correctitud y robustez del compilador frente a un conjunto integral de programas de prueba y casos límite.

Index Terms—Compiladores, Flex, Bison, Código de Tres Direcciones, Ensamblador x86, NASM, EasyLang, Análisis Léxico, Análisis Sintáctico.

I. Introducción

El estudio e implementación de procesadores de lenguajes de programación constituye uno de los pilares formativos y prácticos más rigurosos dentro de la ciencia de la computación y la ingeniería de software. Un compilador tradicional divide su tarea en dos grandes componentes: el Front-End, encargado de validar las reglas léxicas, sintácticas y semánticas del lenguaje fuente para producir una representación intermedia, y el Back-End, responsable de mapear dicha representación al conjunto de instrucciones de la máquina destino.

En el marco del proyecto final de Compiladores, se planteó el desarrollo de un compilador completo para el lenguaje EasyLang (con extensión común .bml), un lenguaje imperativo que combina conceptos sintácticos de lenguajes como Pascal, BASIC y C. Entre sus peculiaridades se encuentran la reserva explícita de variables mediante listas concatenadas por dos puntos seguidas de la palabra clave ALLOCATE, una sintaxis clara para estructuras condicionales e iterativas (FOR, DO-WHILE), soporte para exponenciación nativa (**) y mecanismos básicos de entrada/salida por consola (INPUT y OUTPUT).

El presente artículo documenta la arquitectura técnica, las decisiones de diseño y las diferentes fases que componen la solución implementada:

1. Fase Léxica: Reconocimiento de unidades léxicas (tokens) mediante autómatas finitos deterministas generados con Flex.
2. Fase Sintáctica y Semántica: Especificación gramatical libre de contexto $LALR(1)$ en Bison, acompañada de acciones semánticas embebidas.
3. Generación de Código Intermedio (TDC): Linearización del flujo de control y síntesis de expresiones complejas en cuádruplas/instrucciones de tres direcciones.
4. Generación de Código Objeto (Backend x86): Traductor en C que transforma las directivas intermedias en instrucciones de máquina ensamblador para la arquitectura Intel x86 de 32 bits con sintaxis NASM.

II. Arquitectura Global del Sistema

El flujo de compilación sigue un esquema modular donde cada etapa transforma la información de manera incremental, como se ilustra conceptualmente a continuación:

Pipeline de Compilación:
1. Código Fuente EasyLang (.bml) → lexer.l (Flex).
2. Flujo de Tokens → parser.y (Bison).
3. Código de Tres Direcciones (codigo_tdc.txt) → traductor.c.
4. Código Ensamblador x86 (salida.asm) → NASM + LD → Binario Ejecutable ELF32.

Figura 1. Etapas del pipeline de compilación implementado.

Esta separación estricta entre el generador de código intermedio y el emisor de código ensamblador permite desacoplar la sintaxis de alto nivel de las restricciones

arquitectónicas de la máquina destino, facilitando la depuración y optimización de cada fase.

III. Análisis Léxico con Flex

La fase léxica fue desarrollada en el archivo lexer.l. El analizador es responsable de transformar el flujo de caracteres de entrada en un flujo continuo de componentes léxicos categorizados (tokens), eliminando caracteres no significativos como tabulaciones y retornos de carro, pero preservando los saltos de línea (NEWLINE) como delimitadores gramaticales significativos.

III-A. Categorización de Tokens

En el Cuadro I se resumen las principales categorías de tokens configuradas en el analizador léxico.

Cuadro I
Componentes Léxicos y Tokens Reconocidos

Categoría	Patrón / Expresión Regular	Token Bison
Palabras clave	"MAIN", "BEGIN", .ᴱND"	TK_MAIN, etc.
Control flujo	IF", "THEN", .ᴱLSE"	TK_IF, etc.
Bucles	"FOR", "TO", "DO", "WHILE"	TK_FOR, etc.
Memoria / IO	.ᴬLLOCATE", INPUT", .ᴼUTPUT"	TK_ALLOCATE, etc.
Aritméticos	-", ", "*", /", "**"	TK_SUMA, etc.
Relacionales	-=", «>", «", »", «=", »="	TK_IG, etc.
Lógicos	.ᴬND", .ᴼR"	TK_AND, TK_OR
Asignación	":="	TK_ASIG
Identificador	[a-zA-Z_][a-zA-Z0-9_]*	TK_ID
Constantes	0|[1-9][0-9]*	TK_NUMERO
Comentarios	**([^*]|…)***	TK_COMENTARIO

III-B. Manejo de Comentarios y Fin de Archivo

Un aspecto destacado de la implementación léxica es el soporte para comentarios de bloque delimitados por tres asteriscos (*** comentario ***), implementados mediante una expresión regular que garantiza que los asteriscos intermedios no terminen la coincidencia de forma prematura. Asimismo, se incorporó una regla especial para «EOF» que asegura el envío final de un token NEWLINE antes de llamar a yyterminate(), resolviendo problemas de detección de fin de programa en archivos que carecen de salto de línea al final.

IV. Análisis Sintáctico y Semántico con Bison

El analizador sintáctico (parser.y) implementa una gramática libre de contexto operada mediante el algoritmo de análisis ascendente $LALR(1)$.

IV-A. Estructura del Programa y Gestión de Tipos

Para garantizar la flexibilidad semántica, el tipo semántico (%union) se define con soporte para cadenas (char*), números enteros e información estructurada para el flujo de control:

```
typedef struct {
    char *l_false;
    char *l_end;
    int linea_inicio;
} IfInfo;

typedef struct {
    char *var;
    char *limit;
    char *l_start;
    char *l_end;
    int linea_inicio;
} ForInfo;
```

La estructura de un programa completo exige una cláusula opcional de inicio seguida del bloque principal:

$$\text{programa} \rightarrow \text{opt_inicio} \quad \text{bloque} \tag{1}$$

donde el bloque principal es delimitado por los pares léxicos:

MAIN NEWLINE BEGIN ... END

IV-B. Declaración de Variables (ALLOCATE)

En EasyLang, las variables no requieren de un tipo explícito (se manejan internamente como enteros de 32 bits en la máquina de destino), pero sí de una declaración explícita de reserva. La gramática soporta tanto identificadores únicos como secuencias encadenadas por dos puntos:

```
x:y:z:i:total ALLOCATE;
```

En cada reconocimiento de identificador asociado a un ALLOCATE, la acción semántica genera directivas DECL <id> en el código intermedio, permitiendo que el generador de código posterior reserve la memoria correspondiente en la sección no inicializada (.bss).

IV-C. Estructuras de Control y Traducción Dirigida por la Sintaxis

IV-C1. Sentencia Condicional IF-THEN-ELSE: Para resolver el clásico problema del "else colgante" (dangling else), se empleó la directiva %precedence REDUCE_NO_ELSE junto con la resolución sintáctica de ramas opcionales. En cada sentencia condicional, el encabezado evalúa la condición booleana generando un salto condicional si es falso (ifz) hacia una etiqueta de escape o rama alternativa. Al término del bloque de la rama verdadera, se emite un salto incondicional (jmp) hacia la etiqueta de finalización.

IV-C2. Sentencia Iterativa FOR: La regla FOR encapsula la inicialización de la variable de control, la evaluación del límite en cada ciclo y el incremento unitario automático. Las estructuras semánticas ForInfo almacenan las etiquetas de retorno y salida necesarias para reconstruir el bucle.

IV-C3. Sentencia Iterativa DO-WHILE: La regla DO ... WHILE (cond) emite una etiqueta de inicio antes del cuerpo de instrucciones. Al alcanzar la cláusula WHILE, se evalúa la condición y se genera la instrucción:

ifnz tX goto lY

la cual transfiere el control a la cabecera si el resultado es verdadero.

V. Generación de Código Intermedio (TDC)

La representación intermedia adoptada es el Código de Tres Direcciones (TDC / TAC). En este esquema, las operaciones complejas se descomponen en secuencias linealizadas donde cada instrucción contiene a lo sumo un operador y dos operandos fuente, almacenando el resultado en una variable temporal o destino.

El compilador cuenta con dos generadores de identificadores únicos:

- nuevo_temp(): Genera nombres ordenados de variables temporales (t1, t2, t3, ...).
- nueva_label(): Genera etiquetas de salto simbólico (l1, l2, l3, ...).

V-A. Soporte para Expresiones Aritméticas y Lógicas

Para operaciones aritméticas de dos operandos (+, −, ∗, /, ∗∗), la acción sintáctica sintetiza un nuevo temporal y emite la cuádrupla correspondiente:

$$t_k = op_1 \circ op_2 \tag{2}$$

Para evaluaciones booleanas con operadores lógicos de corto circuito o combinación (AND, OR), el compilador genera temporales que albergan el valor binario resultante (1 o 0), facilitando la toma de decisiones mediante instrucciones condicionales homogéneas (ifz y ifnz).

VI. Traducción a Código Objeto (Backend x86 NASM)

La fase final es desarrollada en traductor.c. Este componente actúa como un transpilador de Código de Tres Direcciones a lenguaje ensamblador para la arquitectura Intel x86 de 32 bits bajo la sintaxis del ensamblador NASM.

VI-A. Asignación y Reutilización de Registros

Para evitar un desbordamiento innecesario de accesos a memoria, el traductor mantiene un registro de estado simple sobre los registros de propósito general eax y ecx.

- temp_eax: Almacena el identificador temporal actualmente cargado en el acumulador eax.
- temp_ecx: Almacena el identificador del registro secundario ecx.

Cuando una instrucción requiere un operando que ya reside en eax, se omite la emisión de una instrucción mov redundante, aplicando una optimización básica a nivel de mirilla (peephole).

VI-B. Traducción de Comparaciones y Lógica

Las comparaciones relacionales ($<, >, \leq, \geq, ==, <>$) se implementan mediante la secuencia típica en arquitecturas CISC x86:

```
    mov eax, [op1]
    cmp eax, [op2]
    setle al          ; segun la operacion
    movzx eax, al     ; extension con ceros a 32
    bits
```

Esto produce un valor booleano canónico (1 o 0) en el registro de 32 bits, el cual es fácilmente evaluable mediante la instrucción test eax, eax seguida de jnz o jz.

VI-C. Estructuración del Archivo Ensamblador

El archivo final generado (salida.asm) contiene dos secciones estándar:

1. Sección .bss: Reserva bloques de memoria no inicializados para cada variable declarada mediante la directiva resd 1 (4 bytes por entero).
2. Sección .text: Define el punto de entrada global _start, seguido de las secuencias de instrucciones traducidas. Concluye con la llamada al sistema sys_exit invocando la interrupción del kernel Linux:

```
    mov eax, 1        ; codigo sys_exit
    xor ebx, ebx      ; codigo de retorno 0
    int 0x80
```

VII. Validación Experimental y Resultados

Para verificar la fidelidad de todo el proceso de compilación, se utilizó la suite de pruebas del directorio codigosprueba/, en particular el programa representativo test_b.bml.

VII-A. Programa Fuente de Prueba

El siguiente fragmento ilustra un programa fuente complejo en EasyLang conteniendo reserva de variables, bucles y condicionales:

```
*** Programa de Prueba EasyLang ***
MAIN
 BEGIN
    x:y:z:i:total ALLOCATE;
    INPUT x;
    INPUT y;
    z:=1+x;

    DO
     total := total + 1;
    WHILE(x < 25 AND x > 0)

    FOR i := 10 TO -1 DO
    BEGIN
     IF (total < 2) THEN BEGIN total := total + i; x
     := 2**z; END
    END

    IF(x < y) THEN
      BEGIN
        z:=x+y;
      END
    ELSE
      BEGIN
        z:=0;
      END
    OUTPUT z;
 END
```

VII-B. Código Intermedio Generado

El compilador genera el archivo codigo_tdc.txt, del cual se muestra a continuación una sección correspondiente a la evaluación del bucle DO-WHILE:

```
l1:
        t2=total+1;
        total=t2;
        t3=x<25;
        t4=x>0;
        t5=t3&&t4;
        ifnz t5 goto l1:
```

VII-C. Código Objeto Ensamblador Generado

El traductor produce el código x86 correspondiente en salida.asm:

```
section .bss
    x resd 1
    y resd 1
    z resd 1
    i resd 1
    total resd 1

section .text
    global _start

_start:
l1:
    mov eax, [total]
    add eax, 1
    mov [total], eax
    mov eax, [x]
    cmp eax, 25
    setl al
    movzx eax, al
    mov ecx, [x]
    cmp ecx, 0
    setg cl
    movzx ecx, cl
    and eax, ecx
    test eax, eax
    jnz l1
```

VII-D. Análisis de Casos Límite (Suite de Trampas)

Se evaluaron los casos especiales presentes en trampa1.txt a trampa7.txt, los cuales pusieron a prueba:

- Anidamiento profundo de condicionales IF-THEN-ELSE.
- Saltos de línea inesperados o ausencia de los mismos al final del archivo.
- Expresiones booleanas compuestas con operadores relacionales mixtos.
- Identificadores no declarados o reutilización de etiquetas en bucles contiguos.

En todas las pruebas válidas, el compilador sintetizó el código intermedio de manera correcta sin incurrir en ambigüedades irresolubles en tiempo de análisis sintáctico.

VIII. Conclusiones y Trabajo Futuro

En este trabajo se demostró el diseño e implementación funcional de un compilador completo para el lenguaje EasyLang utilizando las herramientas estándar Flex y Bison acopladas a un generador de código en ensamblador x86 NASM.

Las principales conclusiones derivadas del proyecto son:

1. Modularidad del pipeline: El uso de Código de Tres Direcciones como puente intermedio simplificó sustancialmente la tarea de traducción final hacia ensamblador, aislando las complejidades gramaticales de las restricciones de la arquitectura x86.
2. Gestión de flujo de control: El empleo de estructuras semánticas en la unión de Bison (IfInfo, ForInfo) permitió un manejo limpio de las etiquetas de salto para bucles y condicionales anidados.
3. Generación de Ensamblador x86: El backend demostró ser eficiente para el conjunto de instrucciones del lenguaje, realizando una asignación estática clara en la sección .bss y mapeando comparaciones booleanas complejas mediante instrucciones canónicas (cmp, setCC, movzx).

Como trabajo futuro se contempla:

- Incorporar un algoritmo de asignación global de registros basado en coloración de grafos (algoritmo de Chaitin-Briggs).
- Implementar soporte para tipos de datos adicionales (flotantes, cadenas y arreglos multidimensionales).
- Desarrollar optimizaciones independientes de la máquina en el código intermedio, tales como eliminación de subexpresiones comunes y propagación de constantes.

Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi, y J. D. Ullman, Compiladores: Principios, Técnicas y Herramientas, 2da ed. Pearson Educación, 2008.

[2] J. R. Levine, T. Mason, y D. Brown, lex & yacc, 2da ed. O'Reilly Media, 1992.

[3] H. P. Anvin et al., NASM - The Netwide Assembler: Manual, versión 2.15, 2020.

[4] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer's Manual, vol. 1: Basic Architecture, 2021.

[5] K. D. Cooper y L. Torczon, Engineering a Compiler, 2da ed. Morgan Kaufmann, 2011.